Final Project for CS 461

Due Date: 12/17/2025 11:59pm ET

1 Overview

In this assignment, you will develop a model to detect sarcasm using the dataset provided here. This is both a learning exercise and a **competition** where your model's performance on a hidden test set will contribute to your grade.

Important Note

Academic Integrity: While you may use AI tools for learning and debugging, the core implementation must be your own work. We have designed this assignment to ensure genuine understanding. All team members must be thoroughly familiar with every aspect of the code and methodology, as you may be questioned individually about your implementation.

1.1 Group Policy

- Groups of 1-3 students are permitted
- **Each team member must understand all code and concepts used**
- You may be individually questioned about any part of the implementation
- Clearly indicate all team members on your report

2 Task Description

You will train a classification model to predict whether a phrase (short text) is sarcastic or not. The dataset contains two columns text and label.

2.1 Constraints and Requirements

1. **NO Transformer-Based Architectures:** You **cannot** use transformer models (BERT, GPT, RoBERTa, etc.) or any attention-based architectures for this task.
2. **Model Flexibility:** You are **free to use any non-transformer model(s)** you want, including but not limited to:
   - Traditional ML models (Logistic Regression, SVM, Random Forests, etc.)
   - Classical neural networks (MLPs, CNNs, RNNs, LSTMs, BiLSTMs)
   - Ensemble methods
   - Any combination of the above

3. **Computational Efficiency:** Your final model must be able to run inference on a standard laptop (reasonable memory and CPU constraints).

4. **Optional Hint:** Consider exploring additional learning techniques, particularly one round of uncertainty/certainty sampling, which might improve your model's performance. This is not required but encouraged.

3 Deliverables

You must submit a **single ZIP file** named: `cs-461_final_project.zip`. Every person should make a submission, regardless of whether you are in a group or not.

3.1 Required Contents

1. **Report (PDF):** `report.pdf`
2. **Inference Code:** `predict_sarcasm.py`
3. **Model Weights:** Saved model file(s) in a `models/` directory (running the inference should not require internet connection.)
4. **Requirements File:** `requirements.txt` (Python dependencies)

3.2 Directory Structure in code

Your submission must follow this structure with **relative paths only** (no absolute paths):

```
cs-461_final_project.zip.zip

 report.pdf
 requirements.txt
 predict_sarcasm.py
 models/
    model_weights.pkl (or .h5, .pt, etc.)
    vectorizer.pkl (if applicable)
 src/ (optional, for additional code)
     utils.py
```

Critical: Path Requirements

- Use **relative paths only** (e.g., `./models/model.pkl`)
- Do NOT use absolute paths (e.g., `/home/user/project/model.pkl`)
- Your code must work on different systems without modification

4 Inference Code Specifications

Your `predict_sarcasm.py` must accept a CSV file and produce predictions as a csv with the text column and an additional new column called prediction

4.1 Required Interface

```
python predict.py --input test_data.csv --output predictions.csv
```

Input format: CSV where at least one column is titled 'text'
Output format: CSV with two columns: `text` and `prediction`. Note: the prediction should be 0 (not sarcasm) or 1 (sarcasm)

Note: The grading script will execute your code on all submissions using this standardized interface.

5 Report Requirements

5.1 Required Sections

1. **Introduction**
    - Problem description
    - Your approach overview
    - Team member contributions
2. **Data Exploration & Preprocessing**
    - Dataset statistics and characteristics
    - Preprocessing steps (cleaning, lowercasing, etc.)
    - Why you chose these preprocessing methods
    - Any data augmentation techniques used
3. **Feature Engineering**
    - Feature extraction methods (TF-IDF, word embeddings, etc.)
    - Justification for your feature choices
    - Feature selection techniques (if any)
4. **Model Architecture & Selection**
    - Models considered and why
    - Final model architecture details
    - Hyperparameter choices and tuning process
    - Why this model suits the task
5. **Training Methodology**
    - Training procedure and optimization
    - Loss function and evaluation metrics
    - Validation strategy (cross-validation, train/val split, etc.)
    - Any regularization techniques used
    - Learning techniques used (if applicable)
6. **Experiments & Results**
    - Baseline model results

- Ablation studies (impact of different components (e.g. hyperparameters, features, etc.)
- Performance on open test set (F1, Precision, Recall, and Accuracy)
- Confusion matrix and error analysis

7. **Discussion**
   - What worked and what didn't
   - Insights about sarcasm detection
   - Model limitations
   - Potential improvements
8. **Conclusion**
   - Summary of findings
   - Key takeaways
9. **References**
   - All papers, libraries, and resources used

5.2 Report Quality Expectations

- Clear, professional writing
- Proper explanations for all methods and tools used
- Well-formatted tables and figures
- Explain **why** you made each decision, not just **what** you did
- Include code snippets where relevant
- Discuss the **impact** of each methodological choice

6 Grading Breakdown

Component	**Weight**
Report Quality	15%
Open Test Set Performance	5%
Hidden Test Set Performance (Competition)	10%
Total	**30%**

Table 1: Grade distribution for the assignment

7 Optional Hints

This section is entirely optional, but may help you improve your model's performance in the competition.

7.1 Avoiding Overfitting

When training your models, make sure you are not overfitting to the training set. In particular:

- you may use K-fold cross validation.
- Monitor both training and validation accuracy (or loss). A large gap (very high training performance but much lower validation performance) is a strong sign of overfitting.
- Consider using simple regularization techniques such as:
  - Limiting model complexity (e.g., fewer features, simpler models).
  - Using L_2 regularization or dropout (if applicable).
  - Early stopping based on validation performance.

A well-regularized model that generalizes to unseen data will typically perform better on the hidden test set used for grading.

7.2 Optional: Uncertainty and Similarity-Based Sampling

You are free to use any additional learning strategy, as long as it does not violate the rules stated earlier (e.g., no transformer-based architectures). The following pipeline is an *optional* idea that may help improve your model's performance on the hidden test set.

7.2.1 Motivation

A non-transformer model (e.g., logistic regression, SVM, Naive Bayes, CNN, etc.) may still struggle in regions of the feature space where the training signal is sparse or ambiguous. One way to improve the decision boundary is to identify examples where the model is least confident and then strengthen the model by adding similar nearby samples to the training set.

This approach combines:

- **Uncertainty sampling**: find examples for which the model is unsure.
- **Similarity-based expansion**: find additional samples that resemble those uncertain examples.

7.2.2 Step-by-Step Pipeline

Below is a suggested procedure you may follow:

1. **Train an initial model.** Train your chosen model on the provided training set (or on an initial subset).
2. **Compute confidence for all training samples.** For each example x, obtain predicted probabilities $p(y \mid x)$. Define an uncertainty score, for example:

   $$u(x) = 1 - \max_y p(y \mid x),$$

   where high $u(x)$ means high uncertainty.
3. **Select the most uncertain samples.** Sort all examples by $u(x)$ in descending order and pick the top k samples that the model is least confident about.
4. **Represent the data in a vector space.** Embed all examples (including the uncertain ones). The goal is to compute similarity between samples.

5. **Find the nearest neighbors of each uncertain sample.** For each uncertain sample x_u, compute similarity (e.g., cosine similarity) to all other examples and choose the top m most similar ones:
$$\text{NN}(x_u) = \{x \mid \text{cosine}(x, x_u) \text{ is high}\}.$$

6. **Create an expanded training set.** Combine:
   - the original labeled data,
   - the k most uncertain samples,
   - their m nearest neighbors.

   All labels come from the dataset (you are not creating or guessing labels).

7. **Retrain your model.** Retrain the model using this expanded dataset. The intention is to make the model more robust in regions where it was previously uncertain.

8. **Evaluate on your validation set.** Compare performance before and after this step. If helpful, you may repeat the process for one additional round.

7.2.3 Notes

- This is only *one* example. You are encouraged to propose and experiment with alternative selection strategies or other learning techniques, as long as they comply with the assignment rules.
- For similarity-based selection, you may use cosine similarity, Euclidean distance, or any other metric supported by your text representation method.
- Keep in mind the risk of **overfitting**. Always evaluate on a separate validation set, monitor generalization, and apply regularization techniques as needed.

This optional pipeline **may** help improve performance in the competition, particularly if the model initially struggles with ambiguous or borderline cases.

8 Submission Guidelines

1. Ensure all paths are relative
2. Test your `predict_sarcasm.py` on the open test set before submission and report F1, Precision, Recall, and Accuracy on the provided test set.
3. Verify your ZIP file extracts correctly

9 FAQ

Q: Can I use pre-trained word embeddings like Word2Vec or GloVe?
A: Yes! Pre-trained embeddings are allowed as long as they're not from transformer models.

Q: Can I use LSTM or BiLSTM?
A: Yes, RNNs and LSTMs are permitted.

Q: What if I want to use multiple models in an ensemble?
A: Ensembles are encouraged! Just ensure inference remains efficient.

Q: How will the competition ranking work?
A: Rankings will be based on performance metrics (accuracy and F1-score) on the hidden test

set. **Q: Can I use external datasets for training?**
A: No, you must use only the provided dataset for training.

Q: What if my code doesn't run on the grading machine?
A: You will receive a 0 for the performance components. Test thoroughly with relative paths!

Good Luck!

This assignment is designed to deepen your understanding of machine learning for NLP tasks. Focus on understanding *why* different approaches work, not just getting them to run. The competition aspect is meant to be fun and motivating, but learning is the primary goal.

Remember: All team members will be held accountable for understanding the entire project!